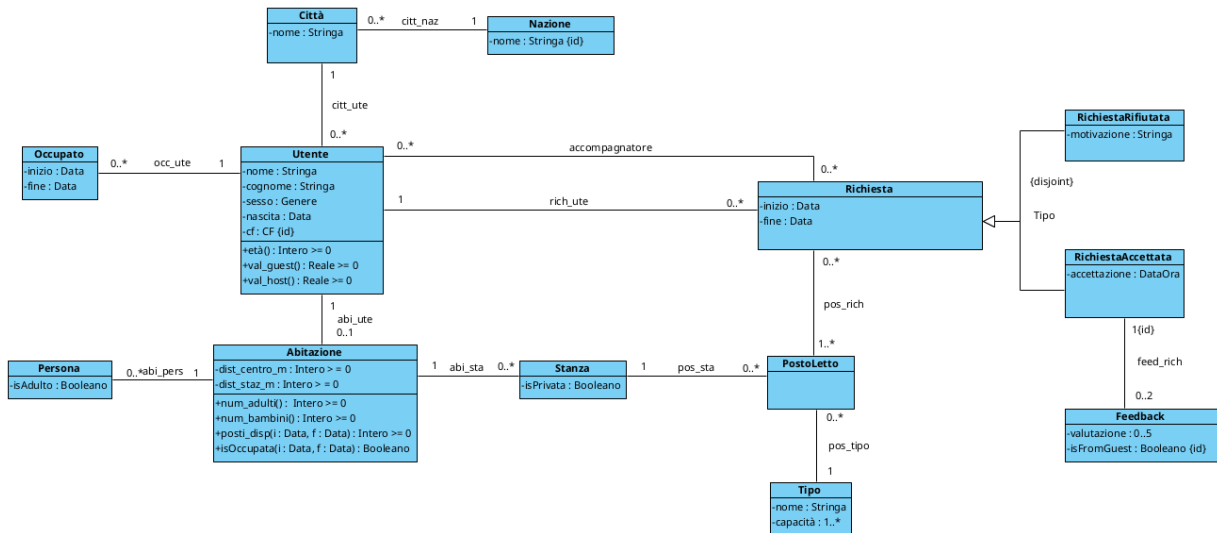


DORMO DA TE

DIAGRAMMA DELLE CLASSI



SPECIFICA DEI TIPI DI DATO

- Genere

{M, F, Altro}

- CF

segue standard codice fiscale

SPECIFICA DELLE CLASSI

- Classe Utente

[V.Utente.no_occupato_prima_nascita]

```
!EXISTS occ, ini, nasc |  
    occ_ute(occ, this) AND  
    inizio(occ, ini) AND  
    nascita(this, nasc) AND  
    ini < nasc
```

[V.Utente.no_nuova_richiesta_senza_feedback_precedente]

```
!EXISTS rich1, fine1, rich2, inizio2 |  
    rich_ute(rich1, this) AND  
    rich_ute(rich2, this) AND  
    rich1 != rich2 AND  
    RichiestaAccettata(rich1) AND  
    fine(rich1, fine1) AND  
    inizio(rich2, inizio2) AND  
    fine1 <= oggi AND  
    inizio2 > fine1 AND  
    !(EXISTS feed | feed_rich(feed, rich1) AND  
        isFromGuest(feed, true))
```

età(): Intero >= 0

pre:

nessuna

post:

result soddisfa la seguente formula:

EXISTS nasc |

nascita(this, nasc) AND

result = oggi - nasc

val_guest(): Reale >= 0

pre:

EXISTS feed, rich |

rich_ute(rich, this) AND

RichiestaAccettata(rich) AND

feed_rich(feed, rich) AND

isFromGuest(feed, false)

post:

result soddisfa la seguente formula:

FeedFromHosts = { f, v | EXISTS rich |

rich_ute(rich, this) AND

RichiestaAccettata(rich) AND

feed_rich(f, rich) AND

isFromGuest(f, false) AND

valutazione(f, v) }

AND

result = SUM(v per ogni (f, v) in FeedFromHosts) /
|FeedFromHosts|

val_host(): Reale >= 0

pre:

```
EXISTS feed, rich, lett, sta, abi |  
    feed_rich(feed, rich) AND  
    RichiestaAccettata(rich) AND  
    isFromGuest(feed, true) AND  
    pos_rich(lett, rich) AND  
    pos_sta(lett, sta) AND  
    abi_sta(abi, sta) AND  
    abi_ute(abi, this)
```

post:

result soddisfa la seguente formula:

```
FeedFromGuests = { f, v | EXISTS rich, pos, sta, abi |  
    feed_rich(f, rich) AND  
    RichiestaAccettata(rich) AND  
    isFromGuest(f, true) AND  
    valutazione(f, v) AND  
    pos_rich(pos, rich) AND  
    pos_sta(pos, sta) AND  
    abi_sta(abi, sta) AND  
    abi_ute(abi, this) }
```

AND

```
result = SUM( v per ogni (f, v) in FeedFromGuests) /  
    |FeedFromGuests|
```

- Classe Occupato

[V.Occupato.no_inizio_dopo_fine]

```
!EXISTS ini, fin |  
    inizio(this, ini) AND
```

```
fine(this, fin) AND  
ini >= fin
```

- Classe Richiesta

[V.Richiesta.no_inizio_dopo_fine]

```
!EXISTS ini, fin |  
    inizio(this, ini) AND  
    fine(this, fin) AND  
    ini >= fin
```

[V.Richiesta.tutti_letti_stessa_abitazione]

```
!EXISTS pos1, pos2, sta1, sta2, abi1, abi2 |  
    pos1 != pos2 AND  
    pos_rich(pos1, this) AND  
    pos_rich(pos2, this) AND  
    pos_sta(pos1, sta1) AND  
    pos_sta(pos2, sta2) AND  
    abi1 != abi2 AND  
    abi_sta(abi1, sta1) AND  
    abi_sta(abi2, sta2)
```

- Classe RichiestaAccettata

[V.RichiestaAccettata.prima_inizio_prenotazione]

```
!EXISTS acc, ini |  
    accettazione(this, acc) AND  
    inizio(this, ini) AND  
    acc >= ini
```

- Classe PostoLetto

[V.PostoLetto.no_due_richiestaAccettata_contemporanee]

```
!EXISTS rich1, ini1, fin1, rich2, ini2, fin2, t |  
    rich1 != rich2 AND  
    RichiestaAccettata(rich1) AND  
    RichiestaAccettata(rich2) AND  
    pos_rich(this, rich1) AND  
    pos_rich(this, rich2) AND  
    inizio(rich1, ini1) AND  
    fine(rich1, fin1) AND  
    inizio(rich2, ini2) AND  
    fine(rich2, fin2) AND  
    Data(t) AND  
    ( t >= ini1 AND t <= fin1) AND  
    ( t >= ini2 AND t <= fin2 )
```

- Classe Abitazione

num_adulti(): Intero >= 0

pre:

nessuna

post:

result soddisfa la seguente formula:

Adulti = { p | abi_pers(this, p) AND
isAdulto(p, true) }

AND

result = |Adulti|

num_bambini(): Intero ≥ 0

pre:

nessuna

post:

result soddisfa la seguente formula:

Bambini = {p | abi_pers(this, p) AND
isAdulto(p, false) }

AND

result = |Bambini|

isOccupata(i:Data, f:Data): Booleano

pre:

nessuna

post:

result soddisfa la seguente formula:

result = (EXISTS u, occ, ini, fin |
abi_ute(this, u) AND
occ_ute(occ, u) AND
inizio(occ, ini) AND
fine(occ, fin) AND
ini $\leq$ f AND i $\leq$ fin)

posti_disp(i:Data, f:Data): Intero ≥ 0

pre:

isOccupata(this, i, f, false)

post:

result soddisfa la seguente formula:

P = { p | EXISTS sta |
pos_sta(p, sta) AND

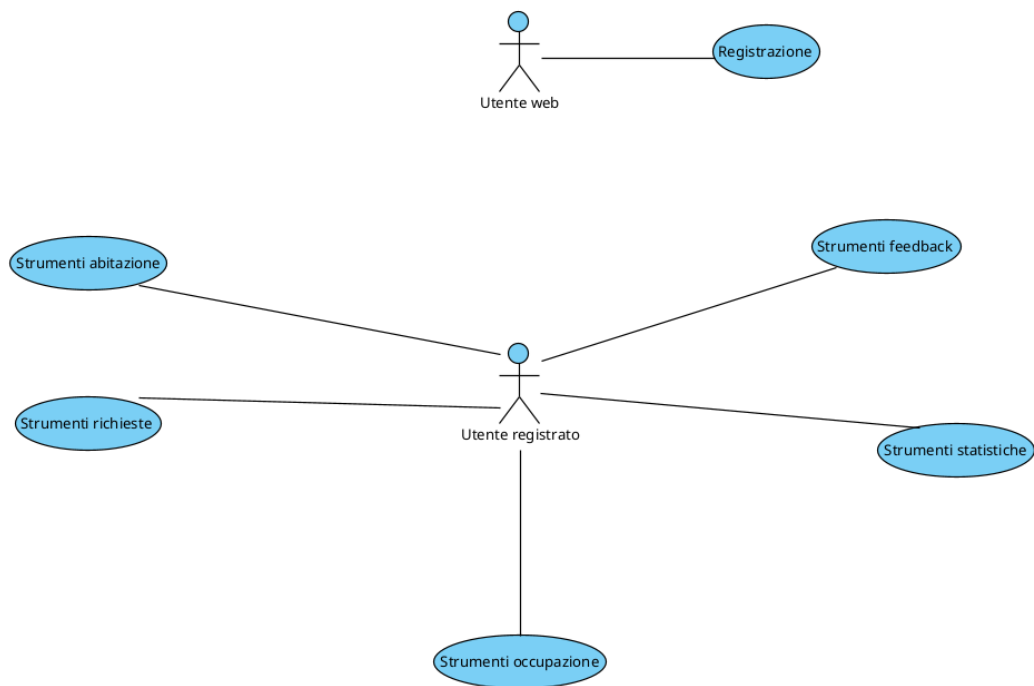
```

abi_sta(this, sta) AND
!EXISTS rich, ini, fin |
    pos_rich(p, rich) AND
    RichiestaAccettata(rich) AND
    inizio(rich, ini) AND
    fine(rich, fin) AND
    ( ini <= f AND i <= fin ) }

AND
result = |P|

```

DIAGRAMMA DEGLI USE-CASE



SPECIFICA DEGLI USE-CASE

- Use-Case Registrazione

**nuovo_utente(n:String, c:String, s:Genere,
nasc:Data, cf:CF, citt:Città): Utente**

pre:

!EXISTS u' |
cf(u', cf)

post:

il livello estensionale finale differisce
da quello iniziale come segue:

nuovi elementi del dominio: alpha

elementi rimossi dal dominio: nessuno

nuove ennuple:

- Utente(alpha)
- nome(alpha, n)
- cognome(alpha, c)
- sesso(alpha, s)
- nascita(alpha, nasc)
- cf(alpha, cf)
- citt_ute(citt, alpha)

ennuple rimosse: nessuna

valore di ritorno: result = alpha

- Use-Case Strumenti abitazione

**collega_abitazione(distcen:Interog=0,
diststaz:Interog=0): Abitazione**

pre:

sia u:Utente l'istanza dell'attore che ha invocato
lo use-case

!EXISTS a' |

abi_ute(a', u)

post:

il livello estensionale finale differisce
da quello iniziale come segue:

nuovi elementi del dominio: alpha

elementi rimossi dal dominio: nessuno

nuove ennuple:

- Abitazione(alpha)

- dist_centro_m(alpha, distcen)

- dist_staz_m(alpha, diststaz)

sia u:Utente l'istanza dell'attore che ha
invocato lo use-case

- abi_ute(alpha, u)

ennuple rimosse: nessuna

valore di ritorno: result = alpha

**aggiungi_familiare_abitazione(abi:Abitazione,
isA:Booleano): Persona**

pre:

sia u:Utente l'istanza dell'attore che ha invocato
lo use-case

abi_ute(abi, u)

post:

il livello estensionale finale differisce
da quello iniziale come segue:

nuovi elementi del dominio: alpha

elementi rimossi dal dominio: nessuno

nuove ennuple:

- Persona(alpha)
- isAdulto(alpha, isA)
- abi_pers(abi, alpha)

ennuple rimosse: nessuna

valore di ritorno: result = alpha

aggiungi_stanza(abi:Abitazione, isP:Booleano): Stanza

pre:

sia u:Utente l'istanza dell'attore che ha invocato
lo use-case

abi_ute(abi, u)

post:

il livello estensionale finale differisce
da quello iniziale come segue:

nuovi elementi del dominio: alpha

elementi rimossi dal dominio: nessuno

nuove ennuple:

- Stanza(alpha)
- abi_sta(abi, alpha)
- isPrivata(alpha, isP)

ennuple rimosse: nessuna

valore di ritorno: result = alpha

aggiungi_postoLetto(sta:Stanza, tip:Tipo): PostoLetto

pre:

sia u:Utente l'istanza dell'attore che ha invocato
lo use-case

EXISTS abi |

abi_sta(abi, sta) AND

abi_ute(abi, u)

post:

il livello estensionale finale differisce
da quello iniziale come segue:

nuovi elementi del dominio: alpha

elementi rimossi dal dominio: nessuno

nuove ennuple:

- PostoLetto(alpha)

- pos_tipo(alpha, tip)

- pos_sta(alpha, sta)

ennuple rimosse: nessuna

valore di ritorno: result = alpha

- Use-Case Strumenti occupazione

nuova_occupazione(i:Data, f:Data): Occupato

pre:

i < f

post:

il livello estensionale finale differisce
da quello iniziale come segue:

nuovi elementi del dominio: alpha

elementi rimossi dal dominio: nessuno

nuove ennuple:

- Occupato(alpha)

sia u:Utente l'istanza dell'attore che ha invocato lo use-case

- occ_ute(alpha, u)

- inizio(alpha, i)

- fine(alpha, f)

ennuple rimosse: nessuna

valore di ritorno: result = alpha

elimina_occupazione(occ:Occupato):

pre:

sia u:Utente l'istanza dell'attore che ha invocato lo use-case

occ_ute(occ, u)

post:

il livello estensionale finale differisce da quello iniziale come segue:

nuovi elementi del dominio: nessuno

elementi rimossi dal dominio: occ

nuove ennuple: nessuna

ennuple rimosse:

- Occupato(occ)

- occ_ute(occ, u)

siano i, f tali che inizio(occ, i) e fine(occ,f)

- inizio(occ, i)

- fine(occ, f)

valore di ritorno: nessuno

- Use-Case Strumenti richieste

```
cerca_abitazioni(  
    cit:Città,  
    i:Data,  
    f:Data,  
    posti:Intero>0  
): Utente[0..*]  
    pre:  
        i < f  
    post:  
        questo use-case non modifica il livello estensionale  
        result soddisfa la seguente formula:  
  
        result = { u | EXISTS abi |  
                    citt_ute(cit, u) AND  
                    abi_ute(abi, u) AND  
                    posti_disp(abi, i, f) >= posti }
```

```
inoltra_richiesta(  
    abi:Abitazione,  
    i:Data,  
    f:Data,  
    posti:PostoLetto[1..*]  
): Richiesta  
    pre:  
        sia u:Utente l'istanza dell'attore che ha invocato  
        lo use-case
```

```

i < f
AND
posti_disp(abi, i, f) >= |posti|
AND
!abi_ute(abi, u)
AND
ALL pos in posti |
    EXISTS sta |
        pos_sta(pos, sta) AND
        abi_sta(abi, sta)
post:
    il livello estensionale finale differisce
    da quello iniziale come segue:

```

```

nuovi elementi del dominio: alpha
elementi rimossi dal dominio: nessuno
nuove ennuple:
    - Richiesta(alpha)
    - rich_ute(alpha, u)
    - inizio(alpha, i)
    - fine(alpha, f)
    - pos_rich(pos, alpha) per ogni pos in posti
ennuple rimosse: nessuna
valore di ritorno: result = alpha

```

accetta_richiesta(ric:Richiesta): RichiestaAccettata

```

pre:
    sia u:Utente l'istanza dell'attore che ha invocato
    lo use-case
    EXISTS lett, sta, abi |
        pos_rich(lett, ric) AND

```

```

        pos_sta(lett, sta) AND
        abi_sta(abi, sta) AND
        abi_ute(abi, u)
    AND
    EXISTS ini |
        inizio(ric, ini) AND
        ini > oggi
    AND
    !( RichiestaAccettata(ric) OR
        RichiestaRifiutata(ric) )
post:
    il livello estensionale finale differisce
    da quello iniziale come segue:

```

```

    nuovi elementi del dominio: nessuno
    elementi rimossi dal dominio: nessuno
    nuove ennuple:
        - RichiestaAccettata(ric)
        - accettazione(ric, adesso)
    ennuple rimosse: nessuna
    valore di ritorno: result = ric

```

rifiuta_richiesta(ric:Richiesta, mot:Stringa): RichiestaRifiutata

```

pre:
    sia u:Utente l'istanza dell'attore che ha invocato
    lo use-case
    EXISTS lett, sta, abi |
        pos_rich(lett, ric) AND
        pos_sta(lett, sta) AND
        abi_sta(abi, sta) AND

```



```

        abi_ute(abi, u)
    AND
    EXISTS ini |
        inizio(ric, ini) AND
        ini > oggi
    AND
    !( RichiestaAccettata(ric) OR
        RichiestaRifiutata(ric) )
post:
    il livello estensionale finale differisce
    da quello iniziale come segue:

```

```

    nuovi elementi del dominio: nessuno
    elementi rimossi dal dominio: nessuno
    nuove ennuple:
        - RichiestaRifiutata(ric)
        - motivazione(ric, mot)
    ennuple rimosse: nessuna
    valore di ritorno: result = ric

```

- Use-Case Strumenti feedback

**rilascia_feed_a_guest(ric:RichiestaAccettata,
v:0..5): Feedback**

```

pre:
    Sia u:Utente l'istanza dell'attore che ha invocato
    lo use-case
    EXISTS let, sta, abi |
        pos_rich(let, ric) AND
        pos_sta(let, sta) AND
        abi_sta(abi, sta) AND

```

```

        abi_ute(abi, u)
    AND
    !EXISTS f' |
        feed_rich(f', ric) AND
        isFromGuest(f', false)
    AND
    EXISTS fin |
        fine(ric, fin) AND
        fin <= oggi
post:

```

il livello estensionale finale differisce
da quello iniziale come segue:

nuovi elementi del dominio: alpha
 elementi rimossi dal dominio: nessuno
 nuove ennuple:

- Feedback(alpha)
- feed_rich(alpha, ric)
- valutazione(alpha, v)
- isFromGuest(alpha, false)

ennuple rimosse: nessuna
 valore di ritorno: result = alpha

rilascia_feed_a_host(ric:RichiestaAccettata, v:0..5): Feedback

```

pre:
    Sia u:Utente l'istanza dell'attore che ha invocato
    lo use-case
    rich_ute(ric, u)
    AND
    !EXISTS f' |

```

```

        feed_rich(f', ric) AND
        isFromGuest(f', true)
    AND
    EXISTS fin |
        fine(ric, fin) AND
        fin <= oggi
post:
    il livello estensionale finale differisce
    da quello iniziale come segue:

    nuovi elementi del dominio: alpha
    elementi rimossi dal dominio: nessuno
    nuove ennuple:
        - Feedback(alpha)
        - feed_rich(alpha, ric)
        - valutazione(alpha, v)
        - isFromGuest(alpha, true)
    ennuple rimosse: nessuna
    valore di ritorno: result = alpha

```

- Use-Case Strumenti statistiche

ottieni_val_guest(u:Utente): Reale $\geq$ 0

pre:

nessuna

post:

questo use-case non modifica il livello estensionale
 result soddisfa la seguente formula:

```
val_guest(u, result)
```

ottieni_val_host(u:Utente): Reale $\geq$ 0

pre:

nessuna

post:

questo use-case non modifica il livello estensionale

result soddisfa la seguente formula:

val_host(u, result)